

TO ESTIMATE THE VALUE OF PI(π) USING MONTE CARLO SIMULATION METHOD.

Initial Conditions:

We assume a 2D coordinate system where a unit circle (radius $r=1$) is inscribed within a square with a side length of 2. The center of both the circle and the square is at the origin (0,0). We assume that a large number of random points (x,y) can be generated within the bounds of the square.

Theory:

The Monte Carlo method relies on the ratio of the areas of the inscribed circle and the bounding square.

- Area of the circle: $A_{circle} = \pi * r^2 = \pi * 1^2 = \pi$
- Area of the square: $A_{square} = (2r)^2 = (2 * 1)^2 = 4$

The ratio of the areas is:

$$\frac{A_{circle}}{A_{square}} = \frac{\pi}{4}$$

By generating N total random points and counting how many points M fall inside the circle (where $x^2 + y^2 \leq 1$), we can estimate the ratio. Therefore:

$$\pi \approx 4 * \frac{M}{N}$$

Procedure:

- Initialize a counter for points falling inside the circle.
- Use a loop to generate a large number (N) of random coordinates x and y between -1 and 1.
- For each point, calculate its distance from the origin using the formula $x^2 + y^2$.
- If $x^2 + y^2 \leq 1$, increment the "inside" counter.
- After the loop, calculate π by multiplying the ratio of "inside" points to total points by 4.

Code:

```
#include <cmath>
#include <cstdlib>
#include <ctime>
#include <iostream>

const int max = RAND_MAX;
time_t t = 0;

double r_point() { return (double)rand() / max * 2.0 - 1.0; }
void estimate_pi(const char *sampleCase, long long iterations) {
    long long inside_circle = 0;
    long long total_points = iterations;
```

```

    for (long long i = 0; i < total_points; ++i) {
        double x = r_point();
        double y = r_point();
        if (x * x + y * y <= 1.0) {
            inside_circle++;
        }
    }
    double pi_estimate = 4.0 * (double)inside_circle / total_points;
    std::cout << "\nSample Case: " << sampleCase << "\n";
    std::cout << "Points Inside Circle: " << inside_circle << "\n";
    std::cout << "Estimated Pi: " << pi_estimate
        << ", With total points = " << total_points << std::endl;
}

int main() {
    srand((unsigned)time(&t));
    int total = 3;
    estimate_pi("1", 1000000);
    estimate_pi("2", 10000000);
    estimate_pi("3", 100000000);
    return 0;
}

```

Output:

```

/mnt/Development/Programming/Collage/fifthSem/SM main*
● > cd "/mnt/Development/Programming/Collage/fifthSem/SM/" &&
  tio-f_pi

Sample Case: 1
Points Inside Circle: 784942
Estimated Pi: 3.13977, With total points = 1000000

Sample Case: 2
Points Inside Circle: 7852446
Estimated Pi: 3.14098, With total points = 10000000

Sample Case: 3
Points Inside Circle: 78532572
Estimated Pi: 3.1413, With total points = 100000000

/mnt/Development/Programming/Collage/fifthSem/SM main*
○ > █

```

Conclusion:

Hence, in this way, the value of π was estimated using the Monte Carlo simulation. The accuracy of the estimation increases as the number of random samples (points) increases.

GENERATE 20 PSEUDO RANDOM NUMBERS USING MIXED CONGRUENTIAL METHOD.

Initial Conditions:

To initiate the generator, four integer parameters are required:

- **Seed (X_0):** The starting value (e.g., $X_0 = 5$).
- **Multiplier (a):** A constant integer (e.g., $a = 13$).
- **Increment (c):** A constant integer (e.g., $c = 7$).
- **Modulus (m):** The upper bound for the numbers (e.g. $m = 100$).

Theory:

The Mixed Congruential Method generates a sequence of random numbers using the linear recurrence relation: $X_n = (a * X_{n-a} + c) \bmod m$

Where:

- X_{n-a} is the next pseudo-random number in the sequence.
- The method is called "Mixed" because it uses both addition ($+ c$) and multiplication ($a *$).
- The sequence will eventually repeat itself with a period no greater than m . For a "Full Period" (*length* m), the parameters must follow Hull-Dobell Theorem conditions.

Procedure:

- Define the constants for a , c , m and the initial seed X_0 .
- Create a loop that executes 20 times to generate the required count of numbers.
- In each iteration, calculate the next value using the formula $(a * X_{n-a} + c) \% m$.
- Update the "current" value with the result of the calculation for the next iteration.
- Display the resulting integer in each step.

Code:

```
#include <iomanip>
#include <iostream>
using ll = long long;

void GenerateLCG(ll M, ll A, ll C, ll X0, int count = 20) {
    ll m = M, a = A, c = C, x0 = X0;
    ll current_x = x0;

    std::cout << "\nGenerating " << count
               << " Pseudo-Random Numbers using LCG:" << std::endl;
    std::cout << "Formula:  $X_{n+1} = (a * X_n + c) \bmod m$ "
               << std::endl;
    std::cout << "-----"
               << std::endl;
```

```

std::cout << std::setw(5) << "n" << std::setw(15) << "X_n" << std::endl;

for (int i = 1; i <= count; ++i) {
    current_x = (a * current_x + c) % m;
    std::cout << std::setw(5) << i << std::setw(15) << current_x
                << std::endl;
}

}

int main() {
    GenerateLCG(100, 13, 7, 5);
    return 0;
}

```

Output:

```

/mnt/Development/Programming/Collage/fifthSem/SM main*
• > cd "/mnt/Development/Programming/Collage/fifthSem/SM/
ge/fifthSem/SM/"Mixed_Congruential_Method

Generating 20 Pseudo-Random Numbers using LCG:
Formula:  $X_{n+1} = (13 * X_n + 7) \bmod 100$ 
-----

```

n	X_n
1	72
2	43
3	66
4	65
5	52
6	83
7	86
8	25
9	32
10	23
11	6
12	85
13	12
14	63
15	26
16	45
17	92
18	3
19	46
20	5

Conclusion:

Hence, in this way, a sequence of 20 pseudo-random numbers was generated using the Mixed Congruential Method.

PERFORM KOLMOGROV-SMIRNOV(KS) TEST TO TEST THE UNIFORMITY OF THE GIVEN RANDOM NUMBERS.

[0.44,0.81,0.14,0.05,0.91]

Initial Conditions:

- Given random numbers: [0.44, 0.81, 0.14, 0.05, 0.91].
- Sample size (n): 5.
- Level of significance (α): 0.05 (assumed).
- Critical value (D_α): For $n = 5$, $\alpha = 0.05$. The critical value is 0.565.

Theory:

The Kolmogorov-Smirnov test compares the empirical distribution function $S_n(x)$ of a sample with the theoretical cumulative distribution function $f(x)$ of the uniform distribution. The test statistic D is calculated as:

$$D = \max(D^+, D^-)$$

Where:

$$D^+ = \frac{\max}{1 \leq i \leq n} \left(\frac{i}{n} - R_{(i)} \right) \quad \text{and} \quad D^- = \frac{\max}{1 \leq i \leq n} \left(R_{(i)} - \frac{i-1}{n} \right)$$

Here, $R_{(i)}$ are the sorted random numbers. If the calculated D is less than or equal to the critical value D_α , the hypothesis of uniformity is accepted.

Procedure:

- Rank the given n random numbers from smallest to largest:
 $R_{(1)} \leq R_{(2)} \leq \dots \leq R_{(n)}$.
- Compute D^+ by finding the maximum value of $\left(\frac{i}{n} - R_{(i)} \right)$ for all i .
- Compute D^- by finding the maximum value of $\left(R_{(i)} - \frac{i-1}{n} \right)$ for all i .
- Determine the overall statistic $D = \max(D^+, D^-)$.
- Compare the calculated D with the critical table value D_α for the given sample size n .
- If $D \leq D_\alpha$, conclude that the numbers are uniformly distributed.

Code:

```
#include <algorithm>
#include <iomanip>
#include <iostream>
#include <vector>

void ksTest(std::vector<double> R, float critical_value) {
    int n = R.size();
    std::sort(R.begin(), R.end());
    std::cout << "Sorted Numbers: ";
    for (double val : R)
        std::cout << val << " ";
    std::cout << "\n\n";
    double max_d_plus = 0;
    double max_d_minus = 0;
    std::cout << std::setw(5) << "i" << std::setw(10) << "Ri" << std::setw(10)
        << "i/n" << std::setw(10) << "D+" << std::setw(10) << "D-"
        << std::endl;

    for (int i = 1; i <= n; ++i) {
        double d_plus = (double)i / n - R[i - 1];
        double d_minus = R[i - 1] - (double)(i - 1) / n;

        if (d_plus > max_d_plus)
            max_d_plus = d_plus;
        if (d_minus > max_d_minus)
            max_d_minus = d_minus;

        std::cout << std::setw(5) << i << std::setw(10) << R[i - 1]
            << std::setw(10) << (double)i / n << std::setw(10) << d_plus
            << std::setw(10) << d_minus << std::endl;
    }

    double D = std::max(max_d_plus, max_d_minus);

    std::cout << "\nMax D+ = " << max_d_plus << std::endl;
    std::cout << "Max D- = " << max_d_minus << std::endl;
    std::cout << "Calculated D = " << D << std::endl;

    std::cout << "Critical Value (D_0.05, 5) = " << critical_value << std::endl;
    if (D <= critical_value) {
        std::cout
            << "Result: Accept H0 (The numbers are uniformly distributed)."
            << std::endl;
    } else {
        std::cout
            << "Result: Reject H0 (The numbers are not uniformly distributed)."
            << std::endl;
    }
}
```

```
int main() {
    std::vector<double> R = {0.44, 0.81, 0.14, 0.05, 0.91};
    ksTest(R, 0.565); // Critical value for n=5 at alpha=0.05
    return 0;
}
```

Output:

```
/mnt/Development/Programming/Collage/fifthSem/SM main*
• > cd "/mnt/Development/Programming/Collage/fifthSem/SM/" && g++ ks_te
Sorted Numbers: 0.05 0.14 0.44 0.81 0.91
```

i	Ri	i/n	D+	D-
1	0.05	0.2	0.15	0.05
2	0.14	0.4	0.26	-0.06
3	0.44	0.6	0.16	0.04
4	0.81	0.8	-0.01	0.21
5	0.91	1	0.09	0.11

```

Max D+ = 0.26
Max D- = 0.21
Calculated D = 0.26
Critical Value (D_0.05, 5) = 0.565
Result: Accept H0 (The numbers are uniformly distributed).
```

Conclusion:

Hence, in this way, the Kolmogorov-Smirnov (KS) test was performed, and the given set of random numbers was found to be uniformly distributed at a 5% significance level.

TO IMPLEMENT MARKOV CHAIN FOR BIKE PURCHASE PREDICTION

Initial Conditions:

We assume a current Honda bike user, meaning the initial state vector is $S_0 = [1, 0]$ (100% chance for Honda, 0% for Yamaha). The transition probabilities are fixed at 70% retention for Honda and 80% retention for Yamaha.

Theory:

A Markov Chain is a stochastic model describing a sequence of possible events in which the probability of each event depends only on the state attained in the previous event. The transition matrix P is:

$$P = \begin{bmatrix} 0.7 & 0.3 \\ 0.2 & 0.8 \end{bmatrix}$$

The probability distribution after n steps is calculated by multiplying the state vector by the transition matrix n times. ($S_n = S_0 * P^n$).

Procedure:

- Define the initial state vector $[1, 0]$.
- Represent the transition probabilities in a 2×2 matrix.
- Use a loop to iterate through three purchase cycles.
- In each iteration, update the probabilities of owning a Honda or Yamaha using the matrix multiplication rules.
- Output the probability of owning a Yamaha bike after the third iteration.

Code:

```
#include <iomanip>
#include <iostream>
#include <vector>

int main() {
    // Transition Matrix P
    // [ P(H|H) P(Y|H) ] -> [ 0.7  0.3 ]
    // [ P(H|Y) P(Y|Y) ] -> [ 0.2  0.8 ]
    double transition[2][2] = {{0.7, 0.3}, {0.2, 0.8}};
    double state[2] = {1.0, 0.0};
    int steps = 3;
    std::cout << "Markov Chain Transitions for Honda User:" << std::endl;
    std::cout << "-----" << std::endl;
    for (int i = 1; i <= steps; ++i) {
        double next_honda =
            state[0] * transition[0][0] + state[1] * transition[1][0];
        double next_yamaha =
            state[0] * transition[0][1] + state[1] * transition[1][1];
```

```

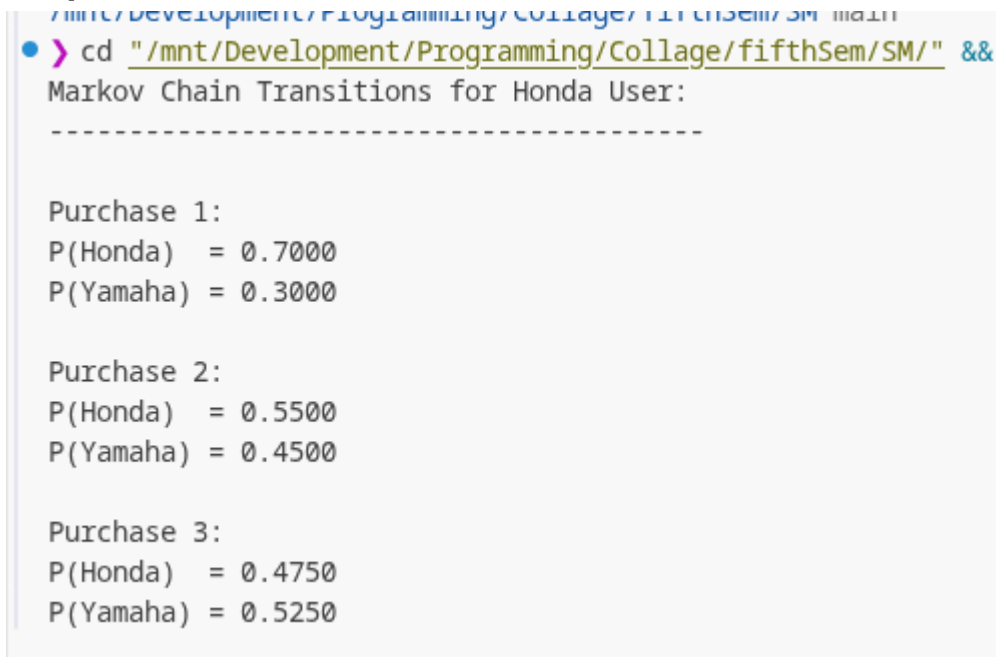
        state[0] = next_honda;
        state[1] = next_yamaha;

        std::cout << "\nPurchase " << i << ":" << std::endl;
        std::cout << "P(Honda)  = " << std::fixed << std::setprecision(4)
                    << state[0] << std::endl;
        std::cout << "P(Yamaha) = " << std::fixed << std::setprecision(4)
                    << state[1] << std::endl;
    }

    return 0;
}

```

Output:



```

/mnt/Development/Programming/Collage/fifthSem/SM main
• > cd "/mnt/Development/Programming/Collage/fifthSem/SM/" &&
Markov Chain Transitions for Honda User:
-----

Purchase 1:
P(Honda)  = 0.7000
P(Yamaha) = 0.3000

Purchase 2:
P(Honda)  = 0.5500
P(Yamaha) = 0.4500

Purchase 3:
P(Honda)  = 0.4750
P(Yamaha) = 0.5250

```

Conclusion:

Hence, in this way the Markov Chain was implemented, and the probability of a current Honda user buying a Yamaha bike after three purchases was found to be 0.525.

TO PERFORM CHI-SQUARE TEST FOR UNIFORMITY ON A SEQUENCE OF RANDOM NUMBERS

Initial Conditions:

- A sequence of $n = 100$ random numbers are provided.
- The numbers are expected to be uniformly distributed between 0 and 1.
- We divide the range $[0, 1]$ into $k = 10$ equal intervals.
- Level of significance (α) is assumed to be 0.05.

Theory:

The Chi-Square test (χ^2) measures the difference between observed frequencies (O_i) and expected frequencies (E_i). For a uniform distribution, the expected frequency for each interval is $E_i = n/k$. The test statistic is calculated as:

$$\chi^2 = \sum_{i=1}^k \frac{(O_i - E_i)^2}{E_i}$$

If the calculated χ^2 value is less than the critical value from the Chi-Square table for $k-1$ degrees of freedom, the null hypothesis (H_0 : the numbers are uniform) is accepted.

Procedure:

- Count the total number of observations (n).
- Divide the data into 10 intervals: $[0.0-0.1)$, $[0.1-0.2)$, ..., $[0.9-1.0]$.
- Determine the observed frequency (O_i) for each interval by tallying the random numbers.
- Calculate the expected frequency (E_i) for each interval (which is $100 / 10 = 10$).
- Use the formula $\sum \frac{(O_i - E_i)^2}{E_i}$ to find the Chi-Square statistic.
- Compare the result with the critical value $\chi^2_{(0.05,9)} = 16.91$.

Code:

```
#include <cmath>
#include <iomanip>
#include <iostream>
#include <vector>

int main() {
    std::vector<double> random_numbers = {
        0.34, 0.83, 0.96, 0.47, 0.79, 0.99, 0.37, 0.72, 0.06, 0.18, 0.90, 0.76,
        0.99, 0.30, 0.71, 0.17, 0.51, 0.43, 0.39, 0.26, 0.25, 0.79, 0.77, 0.17,
        0.23, 0.99, 0.54, 0.56, 0.84, 0.97, 0.89, 0.64, 0.67, 0.82, 0.19, 0.46,
        0.01, 0.97, 0.24, 0.88, 0.87, 0.70, 0.56, 0.56, 0.82, 0.05, 0.81, 0.30,
        0.40, 0.64, 0.44, 0.81, 0.41, 0.05, 0.93, 0.66, 0.28, 0.94, 0.64, 0.47,
```

```

    0.12, 0.94, 0.52, 0.45, 0.65, 0.10, 0.69, 0.96, 0.40, 0.60, 0.21, 0.74,
    0.73, 0.31, 0.37, 0.42, 0.34, 0.58, 0.19, 0.11, 0.46, 0.22, 0.99, 0.78,
    0.39, 0.18, 0.75, 0.73, 0.79, 0.29, 0.67, 0.74, 0.02, 0.05, 0.42, 0.49,
    0.49, 0.05, 0.62, 0.78};

int n = random_numbers.size();
int num_intervals = 10;
double expected_freq = (double)n / num_intervals;
int observed_freq[10] = {0};

for (double num : random_numbers) {
    int interval = (int)(num * 10);
    if (interval > 9)
        interval = 9;
    observed_freq[interval]++;
}

double chi_square_stat = 0;
std::cout << std::fixed << std::setprecision(2);
std::cout << "Interval | Oi | Ei | (Oi-Ei)^2 / Ei" << std::endl;
std::cout << "-----" << std::endl;

for (int i = 0; i < 10; i++) {
    double term = pow(observed_freq[i] - expected_freq, 2) / expected_freq;
    chi_square_stat += term;
    std::cout << i << "          | " << observed_freq[i] << " | "
                << expected_freq << " | " << term << std::endl;
}

std::cout << "\nCalculated Chi-Square Statistic: " << chi_square_stat
            << std::endl;
std::cout << "Degrees of Freedom (k-1): 9" << std::endl;
std::cout << "Critical Value (alpha=0.05, df=9): 16.91" << std::endl;

if (chi_square_stat <= 16.91)
    std::cout << "Conclusion: Numbers are uniformly distributed."
                << std::endl;
else
    std::cout << "Conclusion: Numbers are NOT uniformly distributed."
                << std::endl;

return 0;
}

```

Output:

```
/mnt/Development/Programming/Collage/fifthSem/SM main*
● > cd "/mnt/Development/Programming/Collage/fifthSem/SM/" && g++ chi_te:
Interval | Oi | Ei | (Oi-Ei)^2 / Ei
-----
0        | 7  | 10.00 | 0.90
1        | 9  | 10.00 | 0.10
2        | 8  | 10.00 | 0.40
3        | 9  | 10.00 | 0.10
4        | 14 | 10.00 | 1.60
5        | 7  | 10.00 | 0.90
6        | 10 | 10.00 | 0.00
7        | 15 | 10.00 | 2.50
8        | 9  | 10.00 | 0.10
9        | 12 | 10.00 | 0.40

Calculated Chi-Square Statistic: 7.00
Degrees of Freedom (k-1): 9
Critical Value (alpha=0.05, df=9): 16.91
Conclusion: Numbers are uniformly distributed.

/mnt/Development/Programming/Collage/fifthSem/SM main*
```

Conclusion:

Hence, in this way, the Chi-Square test was performed on the given sequence, and the random numbers were found to be uniformly distributed.